

0484

Bases de datos

Clase 4

04

Consultas a bases de datos



El lenguaje SQL

- Para **definir la estructura** de la BD:
DDL (Data Definition Language)
 - Definición de tablas y otros objetos de la BD
 - Ejemplo: CREATE TABLE....
- Para **consultar y manipular** la BD:
DML (Data Manipulation Language)
 - Consultas de datos
 - Actualización, inserción y eliminación de datos
- Para **controlar el acceso a los datos** DCL (Data Control Language)
 - Controlar la seguridad de los datos y cuentas de usuario

SQL: Instrucción SELECT

Consulta de datos: **SELECT**

Notación:

SELECT <nombre_cols>

FROM <nombre_tablas>;

Ejemplo sobre la tabla **trabajadores**:

trabajadores (dni, nombre, ciudad, antigüedad, salario, departamento)

SELECT nombre, salario FROM trabajadores;

SQL: Instrucción SELECT

Tabla trabajadores, con la que haremos los ejemplos. para crear e introducir los datos.

**SELECT * FROM
trabajadores;**

	dni	nombre	ciudad	antigüedad	salario	departamento
▶	11112222A	Rojo Iglesias, Marta	Barcelona	12	60000.00	2
	11112222C	Gomez Corachán, Manuel	Madrid	15	22000.00	1
	11112233A	Perez Carrillo, Iván	Bilbao	5	48000.00	2
	11112244B	Torres Marqués, Fernando	Madrid	11	55000.00	2
	11112255B	Rubio Sánchez, María	Sevilla	4	36000.00	3
	11112266C	Llamas Rocasolano, Isabel	Barcelona	13	28000.00	3
	22112222A	Roca Benítez, Elena	Sevilla	6	35000.00	4
	33112222C	Muñoz Casas, Montse	NULL	7	40000.00	5

SQL: Instrucción SELECT

Para mostrar todos los campos (columnas) de la tabla trabajadores:

```
SELECT * FROM trabajadores;
```

Para evitar mostrar filas duplicadas:

```
SELECT DISTINCT nombre FROM trabajadores;
```

Para mostrar sólo 3 registros empezando por el segundo:

```
SELECT * FROM trabajadores LIMIT 2,3;
```

	dni	nombre	ciudad	antigüedad	salario	departamento
▶	11112233A	Perez Carrillo, Iván	Bilbao	5	48000.00	2
	11112244B	Torres Marqués, Fernando	Madrid	11	55000.00	2
	11112255B	Rubio Sánchez, María	Sevilla	4	36000.00	3

	departamento
▶	2
	1
	2
	2
	3
	3
	4
	5

```
SELECT departamento FROM trabajadores;  
SELECT DISTINCT departamento FROM trabajadores;
```

	departamento
▶	2
	1
	3
	4
	5

Filtros (WHERE). Sirven para aplicar condiciones (o restricciones) en la selección de datos.

Notación:

```
SELECT <nombre_cols> FROM  
<nombre_tablas>  
WHERE <condiciones_booleanas>;
```

- Ejemplo:

```
SELECT nombre, dni FROM trabajadores WHERE antigüedad > 10;
```

	nombre	dni	antigüedad
▶	Rojo Iglesias, Marta	11112222A	12
	Gomez Corachán, Manuel	11112222C	15
	Torres Marqués, Fernando	11112244B	11
	Llamas Rocasolano, Isabel	11112266C	13
*	NULL	NULL	NULL

Operadores de comparación aplicables en la cláusula WHERE:

=

>

<

>=

<=

<>

Operadores lógicos aplicables en la cláusula WHERE:

AND / OR / NOT

Otros predicados aplicables en la cláusula WHERE:

<atributo> **BETWEEN** <limit_1>**AND** <limit_2> {Rango de valores}

<atributo> **LIKE** <expr> {cadena de caracteres } Comodines: '%', '_'

<atributo> **IS [NOT] NULL** {consultar si el atributo tiene valor o no}

Ejemplos:

AND, OR:

SELECT * FROM trabajadores WHERE ciudad = "BARCELONA" OR antigüedad >=10;

SELECT * FROM trabajadores WHERE departamento = 3 AND antigüedad >=10;

	dni	nombre	ciudad	antigüedad	salario	departamento
▶	11112266C	Llamas Rocasolano, Isabel	Barcelona	13	28000.00	3
	NULL	NULL	NULL	NULL	NULL	NULL

BETWEEN:

SELECT * FROM trabajadores WHERE salario BETWEEN 20000 AND 40000;

	dni	nombre	ciudad	antigüedad	salario	departamento
▶	11112222C	Gomez Corachán, Manuel	Madrid	15	22000.00	1
	11112255B	Rubio Sánchez, María	Sevilla	4	36000.00	3
	11112266C	Llamas Rocasolano, Isabel	Barcelona	13	28000.00	3
	22112222A	Roca Benítez, Elena	Sevilla	6	35000.00	4
	33112222C	Muñoz Casas, Montse	NULL	7	40000.00	5
	NULL	NULL	NULL	NULL	NULL	NULL

LIKE:

SELECT * FROM trabajadores WHERE ciudad LIKE "B%";

	dni	nombre	ciudad	antiquedad	salario	departamento
▶	11112222A	Rojo Iglesias, Marta	Barcelona	12	60000.00	2
	11112233A	Perez Carrillo, Iván	Bilbao	5	48000.00	2
	11112266C	Llamas Rocasolano, Isabel	Barcelona	13	28000.00	3
	NULL	NULL	NULL	NULL	NULL	NULL

SELECT * FROM trabajadores WHERE ciudad LIKE "_A%";

	dni	nombre	ciudad	antiquedad	salario	departamento
▶	11112222A	Rojo Iglesias, Marta	Barcelona	12	60000.00	2
	11112222C	Gomez Corachán, Manuel	Madrid	15	22000.00	1
	11112244B	Torres Marqués, Fernando	Madrid	11	55000.00	2
	11112266C	Llamas Rocasolano, Isabel	Barcelona	13	28000.00	3
	NULL	NULL	NULL	NULL	NULL	NULL

Operadores en SQL

Operador IN y NOT IN

SELECT * FROM trabajadores WHERE departamento IN (2,3);

	dni	nombre	ciudad	antiquedad	salario	departamento
▶	11112222A	Rojo Iglesias, Marta	Barcelona	12	60000.00	2
	11112233A	Perez Carrillo, Iván	Bilbao	5	48000.00	2
	11112244B	Torres Marqués, Fernando	Madrid	11	55000.00	2
	11112255B	Rubio Sánchez, María	Sevilla	4	36000.00	3
	11112266C	Llamas Rocasolano, Isabel	Barcelona	13	28000.00	3

SELECT * FROM trabajadores WHERE ciudad NOT IN ("barcelona", "bilbao");

	dni	nombre	ciudad	antiquedad	salario	departamento
▶	11112222C	Gomez Corachán, Manuel	Madrid	15	22000.00	1
	11112244B	Torres Marqués, Fernando	Madrid	11	55000.00	2
	11112255B	Rubio Sánchez, María	Sevilla	4	36000.00	3
	22112222A	Roca Benítez, Elena	Sevilla	6	35000.00	4

Tratamiento de los nulos

Tratamiento del valor null, ver los registros que tienen un valor NULL o los que no lo tienen

SELECT * FROM trabajadores WHERE ciudad IS NULL;

	dni	nombre	ciudad	antigüedad	salario	departamento
▶	33112222C	Muñoz Casas, Montse	NULL	7	40000.00	5

SELECT * FROM trabajadores WHERE ciudad IS NOT NULL;

	dni	nombre	ciudad	antigüedad	salario	departamento
▶	11112222A	Rojo Iglesias, Marta	Barcelona	12	60000.00	2
	11112222C	Gomez Corachán, Manuel	Madrid	15	22000.00	1
	11112233A	Perez Carrillo, Iván	Bilbao	5	48000.00	2
	11112244B	Torres Marqués, Fernando	Madrid	11	55000.00	2
	11112255B	Rubio Sánchez, María	Sevilla	4	36000.00	3
	11112266C	Lamas Rocasolano, Isabel	Barcelona	13	28000.00	3
	22112222A	Roca Benítez, Elena	Sevilla	6	35000.00	4

Ordenación de las consultas

Ordenación de los datos presentados (ORDER BY)

Notación:

SELECT

<nombre_cols>

FROM

<nombre_tablas>

[WHERE <condiciones_booleanas>]

ORDER BY <atributo_1>, ..., <atributo_N>;

SELECT nombre, antigüedad

FROM trabajadores

ORDER BY antigüedad DESC; {por defecto es ASC}

	nombre	antigüedad
►	Gomez Corachán, Manuel	15
	Llamas Rocasolano, Isabel	13
	Rojo Iglesias, Marta	12
	Torres Marqués, Fernando	11
	Muñoz Casas, Montse	7
	Roca Benítez, Elena	6
	Perez Carrillo, Iván	5
	Rubio Sánchez, María	4

Ejemplos de ordenación

Crear y ejecutar el comando SELECT para obtener los nombres (ordenados alfabéticamente) de los trabajadores de Barcelona que tengan mas de 10 años de antigüedad.

```
SELECT nombre, ciudad, antigüedad FROM trabajadores  
WHERE ciudad = "barcelona" AND antigüedad > 10 ORDER BY nombre;
```

	nombre	ciudad	antigüedad
►	Llamas Rocasolano, Isabel	Barcelona	13
	Rojo Iglesias, Marta	Barcelona	12

También se puede ordenar indicando el orden de la columna.

```
SELECT nombre, antigüedad, ciudad  
FROM trabajadores  
ORDER by 3 desc, 1 asc;
```

	nombre	antigüedad	ciudad
►	Roca Benítez, Elena	6	Sevilla
	Rubio Sánchez, María	4	Sevilla
	Gomez Corachán, Manuel	15	Madrid
	Torres Marqués, Fernando	11	Madrid
	Perez Carrillo, Iván	5	Bilbao
	Llamas Rocasolano, Isabel	13	Barcelona
	Rojo Iglesias, Marta	12	Barcelona
	Muñoz Casas, Montse	7	NULL

Campos calculados

Campos calculados: podemos utilizar operadores AVG, COUNT, SUM, MAX, MIN o bien hacer operaciones matemáticas entre campos y asignar un nombre (alias).

```
SELECT avg(salario) FROM trabajadores  
WHERE ciudad ="barcelona";
```

	avg(salario)
▶	44000.000000

```
SELECT max(salario) as "SalarioTope"  
FROM trabajadores
```

	SalarioTope
▶	60000.00

```
SELECT nombre, salario/12 FROM trabajadores;
```

```
SELECT nombre, salario/12 as "Mensual"  
FROM trabajadores;
```

	nombre	Mensual
▶	Rojo Iglesias, Marta	5000.000000
	Gomez Corachán, Manuel	1833.333333
	Perez Carrillo, Iván	4000.000000
	Torres Marqués, Fernando	4583.333333
	Rubio Sánchez, María	3000.000000
	Llamas Rocasolano, Isabel	2333.333333
	Roca Benítez, Elena	2916.666667
	Muñoz Casas, Montse	3333.333333

Funciones de cadena.

Funciones de cadena

<http://dev.mysql.com/doc/refman/5.7/en/string-functions.html>

Función	Resultado
LEFT	Obtiene N caracteres empezando por la izquierda.
RIGHT	Obtiene N caracteres empezando por la derecha.
LENGHT	Devuelve la longitud de la cadena
TRIM	Elimina los espacios no significativos, delante y detrás de la cadena.
SUBSTR	Obtiene una subcadena a partir de una determinada posición.
LOWER	Transforma la cadena a minúsculas
UPPER	Transforma la cadena a mayúsculas

Funciones de cadena.

Funciones de cadena

<http://dev.mysql.com/doc/refman/5.7/en/string-functions.html>

```
SELECT nombre, left(nombre,5), right(nombre,5), length(nombre)FROM  
TRABAJADORES;
```

	nombre	left(nombre,5)	right(nombre,5)	length(nombre)
►	Rojo Iglesias, Marta	Rojo	Marta	20
	Gomez Corachán, Manuel	Gomez	anuel	23
	Perez Carrillo, Iván	Perez	Iván	21
	Torres Marqués, Fernando	Torre	nando	25
	Rubio Sánchez, María	Rubio	María	22
	Llamas Rocasolano, Isabel	Llama	sabel	25
	Roca Benítez, Elena	Roca	Elena	20
	Muñoz Casas, Montse	Muñoz	ntse	20

Las fechas se introducen utilizando el formato 'aaaa/mm/dd'.
Algunas funciones interesantes.

- **now()** nos devuelve la fecha y hora actual.
- **datediff()** la diferencia entre dos fechas en dias.
- **year()** para obtener el año de una fecha
- **month()** para obtener el mes de una fecha.
- **monthname()** El nombre del mes.
- **day()** para obtener el día de una fecha.
- **curdate()** devuelve la fecha actual
- **timestamdiff()** diferencia entre fechas y horas.

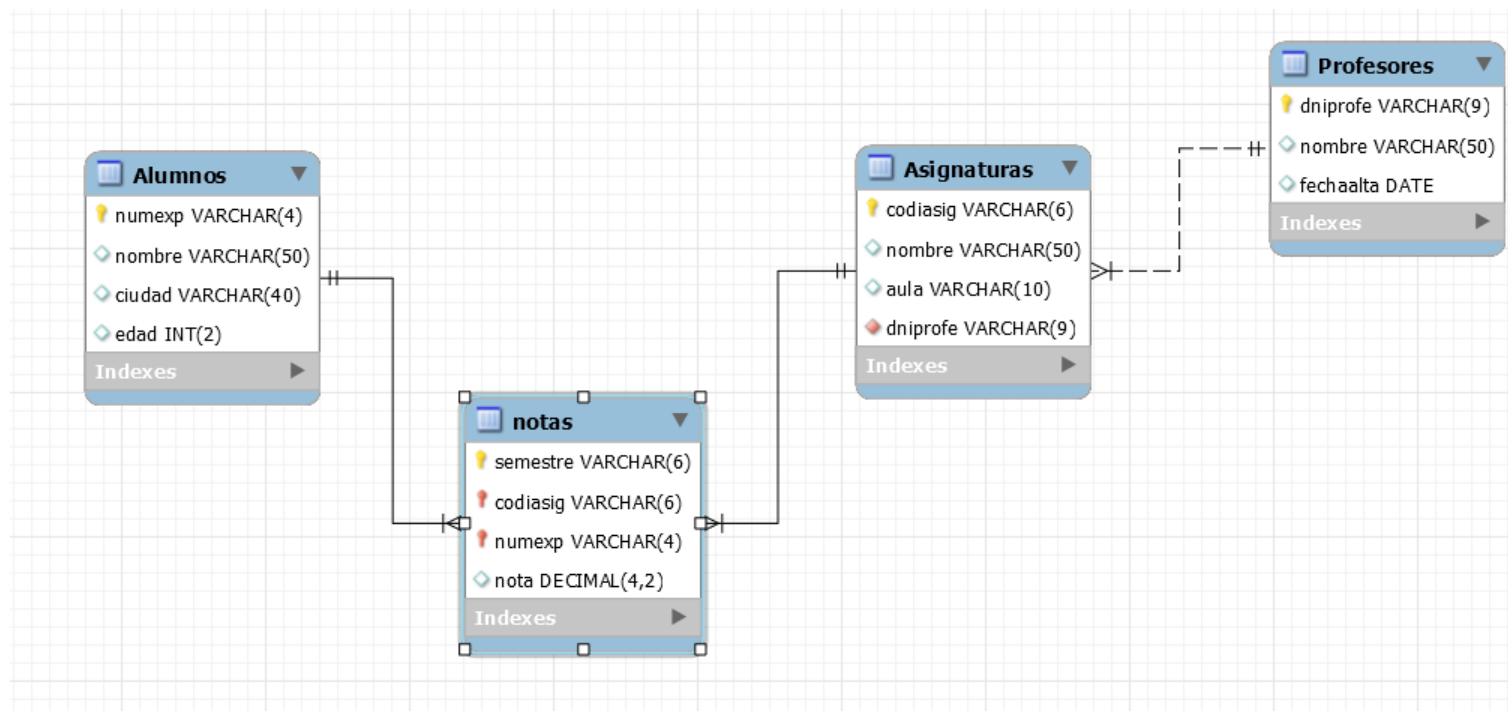
nombre	dias
Rojo Iglesias, Marta	9056
Martinez Camino, José	7890
Perez Carrillo, Iván	8419
Torres Marqués, Fernando	8419
Rubio Sánchez, María	9371
Llamas Rocasolano, Isabel	9060
Gomez Corachán, Manuel	9526

```
select nombre, datediff(now(),fechaalta) as dias
from trabajadores2;

select nombre, year(fechaalta), month(fechaalta),
monthname(fechaalta), day(fechaalta) from trabajadores2;
```

nombre	year(fechaalta)	month(fechaalta)	monthname(fechaalta)	day(fechaalta)
Rojo Iglesias, Marta	1995	1	January	14
Martinez Camino, José	1998	3	March	25
Perez Carrillo, Iván	1996	10	October	12
Torres Marqués, Fernando	1996	10	October	12
Rubio Sánchez, María	1994	3	March	5
Llamas Rocasolano, Isabel	1995	1	January	10

Base de datos escuela.



SELECT con mas de una tabla

Consultas en más de una tabla

Notación:

```
SELECT <nom_cols> FROM <nombre_tabla1>
```

```
INNER JOIN <nombre_tabla2>
```

```
ON tabla1.pk=tabla2.fk;
```

La utilización de inner join, es “más eficiente” que la de seleccionar las dos tablas, ya que solo muestra las filas que tengan elementos asociados.

Ejemplo:

```
select asignaturas.nombre, asignaturas.aula, profesores.nombre  
from asignaturas  
inner join profesores  
on asignaturas.dniprofe=profesores.dniprofe;
```

```
select a.nombre, a.aula, p.nombre  
from asignaturas a  
inner join profesores p  
on a.dniprofe=p.dniprofe;
```

	nombre	aula	nombre
►	Inglés	12	Valle Roca, Ramon
	Matemáticas	10	Garcia Gomez, Jose
	Naturales	11	Garcia Gomez, Jose
	Sociales	14	Martin Ayuso, Montse

Ejemplo 1.

Alumnos y notas de la asignatura de inglés.

```
SELECT a.nombre, n.nota, s.nombre
FROM asignaturas s, notas n, alumnos a
WHERE a.numexp=n.numexp AND n.codiasig=s.codiasig AND
s.nombre="inglés";
```

	nombre	nota	nombre
►	Rojos Iglesias, Marta	9.30	Inglés
	Perez Carrillo, Iván	4.50	Inglés
	Torres Marqués, Fernando	8.20	Inglés
	Rubio Sánchez, María	7.50	Inglés
	Llamos Rocasolano, Isabel	4.00	Inglés
	Comas Prieto, Marta	6.00	Inglés
	Gutierrez Saez, Jose	9.00	Inglés
	Marcos Rico, Fernando	9.70	Inglés
	Villa Bueno, Sandra	2.30	Inglés
	Tebar Mateos, Eva	6.50	Inglés

Ejemplo 2.

Que asignaturas ha suspendido cada alumno.

```
SELECT a.nombre, n.nota, s.nombre
FROM asignaturas s, notas n, alumnos a
WHERE a.numexp=n.numexp AND n.codiasig=s.codiasig AND
n.nota<5;
```

	nombre	nota	nombre
►	Perez Carrillo, Iván	4.50	Inglés
	Llamos Rocasolano, Isabel	4.00	Inglés
	Villa Bueno, Sandra	2.30	Inglés
	Torres Marqués, Fernando	4.20	Matemáticas
	Llamos Rocasolano, Isabel	3.00	Matemáticas
	Villa Bueno, Sandra	4.30	Matemáticas
	Perez Carrillo, Iván	2.50	Naturales
	Torres Marqués, Fernando	3.20	Naturales
	Llamos Rocasolano, Isabel	4.00	Naturales
	Perez Carrillo, Iván	2.50	Sociales

SELECT con mas de una tabla: JOIN, INNER JOIN,

JOIN o INNER JOIN es equivalente a especificar las tablas separadas por comas detrás del FROM y también se sustituye la clausula WHERE por la clausula ON , en tablas muy grandes es mas eficiente ya que utiliza los índices que se hayan creado.

Ejemplo:

```
SELECT a.nombre, a.aula, p.nombre FROM asignaturas a  
INNER JOIN profesores p  
ON a.dniprofe=p.dniprofe;
```

	nombre	aula	nombre
▶	Inglés	12	Valle Roca, Ramon
	Matemáticas	10	Garcia Gomez, Jose
	Naturales	11	Garcia Gomez, Jose
	Sociales	14	Martin Ayuso, Montse

SELECT con mas de una tabla: LEFT JOIN, RIGHT JOIN

Cuando realizamos JOIN de dos tablas todos los registros que no tienen un registro asociado en la segunda tabla no aparecen, para evitar este problema podemos utilizar el LEFT JOIN o el RIGHT JOIN. Con el LEFT van a aparecer todos los registros de la tabla de la izquierda tengan o no valores relacionados en la de la derecha. Y con el RIGHT al revés todos los de la derecha tengan o no relacionados registros en la de la izquierda.

Ejemplo:

```
SELECT a.nombre, a.aula, p.nombre FROM asignaturas a  
RIGHT JOIN profesores p  
ON a.dniprofe=p.dniprofe;
```

Con este ejemplo comprobamos que los profesores que no tienen asignado ninguna asignatura también aparecen en el resultado.

	nombre	aula	nombre
►	Inglés	12	Valle Roca, Ramon
	Matemáticas	10	Garcia Gomez, Jose
	Naturales	11	Garcia Gomez, Jose
	Sociales	14	Martin Ayuso, Montse
	NULL	NULL	Royes Zarate, Jordi

Funciones de agregado.

Permiten obtener un único valor como resultado de aplicar una operación a los valores contenidos en campos.

Las operaciones pueden ser:

- AVG -> Media de los valores de la colección.
- MAX -> devuelve el valor máximo.
- MIN -> devuelve el valor mínimo.
- SUM -> devuelve la suma
- COUNT -> devuelve el número de elementos que tiene la colección.

GROUP BY. Agrupamiento de filas.

Las funciones de agregado trabajan con todos los registros de la tabla.

Pero si utilizamos la cláusula GROUP BY podemos formar grupos de filas de acuerdo con un determinado criterio.

Ejemplos:

Queremos obtener el salario medio por ciudad.

```
select ciudad, avg(salario) from trabajadores  
group by ciudad;
```

	ciudad	avg(salario)
▶	Barcelona	44000.000000
	Bilbao	48000.000000
	Madrid	38500.000000
	Sevilla	35500.000000

Queremos contar el número de trabajadores de cada departamento.

```
select departamento, count(*) from trabajadores  
group by departamento;
```

	departamento	count(*)
▶	1	1
	2	3
	3	2
	4	1
	5	1

HAVING. Filtrar agrupaciones.

Se utiliza para filtrar los resultados de una función de agregado cuando hay agrupamiento de filas. Siempre después de un GROUP BY.

Ejemplos:

Queremos saber de que ciudades tenemos mas de un trabajador.

```
select ciudad, count(*) from trabajadores  
group by ciudad  
having count(*)>1;
```

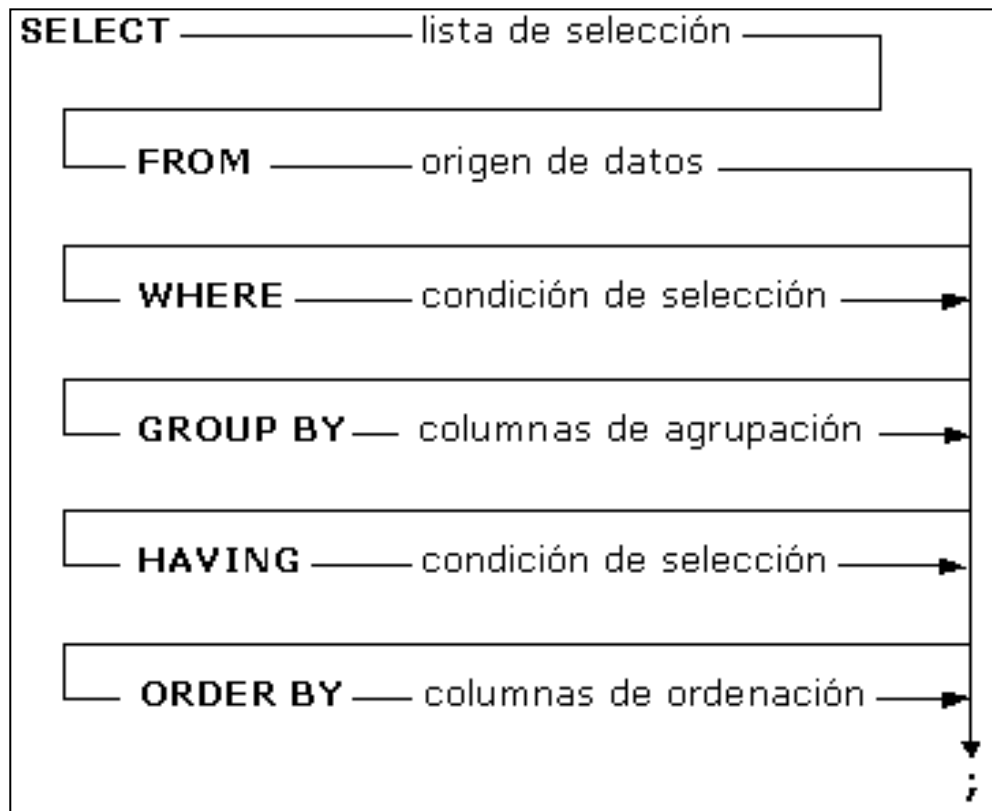
	ciudad	count(*)
▶	Barcelona	2
	Madrid	2
	Sevilla	2

Queremos saber de que ciudades el salario medio es menor de 40000.

```
select ciudad, avg(salario) from trabajadores  
group by ciudad  
having avg(salario)<40000;
```

	ciudad	avg(salario)
▶	Madrid	38500.000000
	Sevilla	35500.000000

Resumen de la sintaxis.



El lenguaje SQL: Subconsultas

Una subconsulta es una orden SELECT que está dentro de otra orden SELECT. Se utiliza para obtener valores basados en valores condicionales desconocidos. A la consulta más interna se le llama subconsulta, ésta devuelve un valor que será utilizado por la consulta principal. Usar una subconsulta equivale a realizar dos consultas secuencialmente y utilizar el resultado de la subconsulta como valor para la consulta principal.

Sintaxis:

SELECT lista campos FROM tabla WHERE
campo operador (SELECT campo FROM tabla)

El lenguaje SQL: Subconsultas

- La subconsulta debe ir entre paréntesis.
- Una subconsulta debe aparecer a la derecha del operador.
- Los campos que comparamos en la consulta y en la subconsulta han de ser los mismos o por lo menos del mismo tipo.
- Las subconsultas pueden devolver un valor o más de un valor. Si devuelven un valor utilizaremos operadores monoregistro, si devuelven más de un valor utilizaremos operadores multiregistro.

SELECT lista campos FROM tabla WHERE
campo operador (SELECT campo FROM tabla)

Subconsultas: monoregistro.

Las subconsultas monoregistro son aquellas que la consulta interna devuelve un solo valor. Para estas consultas utilizaremos operadores monoregistro: =, <, >, >=, <=.

Ejemplo 1.

Listar los trabajadores que tienen mas antigüedad que Torres marqués.

```
SELECT nombre, antigüedad FROM trabajadores  
WHERE antigüedad > (SELECT antigüedad FROM trabajadores  
WHERE nombre like "Torres Marqués%");
```

	nombre	antigüedad
►	Rojo Iglesias, Marta	12
	Gomez Corachán, Manuel	15
	Llamas Rocasolano, Isabel	13

Subconsultas: multiregistro.

Las subconsultas multiregistro son aquellas que la consulta interna puede devolver más de un valor. Para estas consultas utilizaremos operadores multiregistro, que son:

IN (igual que uno de los valores), ALL (que todos los valores) y ANY (uno de los valores).

Ejemplo 2.

Ejemplo ALL se cumplen todas las condiciones. Listar todos los trabajadores que cobren mas que todos los trabajadores del departamento 3.

`SELECT nombre, salario FROM trabajadores
WHERE salario > ALL (SELECT salario from trabajadores where
departamento=3);`

Resultado de
la subconsulta

	salario
▶	36000.00
	28000.00

	nombre	salario
▶	Rojo Iglesias, Marta	60000.00
	Perez Carrillo, Iván	48000.00
	Torres Marqués, Fernando	55000.00
	Muñoz Casas, Montse	40000.00

Subconsultas: multiregistro.

IN (igual que uno de los valores), ALL (que todos los valores) y ANY (uno de los valores).

SELECT nombre, salario FROM trabajadores
WHERE salario IN (SELECT salario from trabajadores where departamento=3);

	nombre	salario	departamento
▶	Rubio Sánchez, María	36000.00	3
	Llamas Rocasolano, Isabel	28000.00	3

SELECT nombre, salario FROM trabajadores
WHERE salario > ANY (SELECT salario from trabajadores where departamento=3);

	nombre	salario	departamento
▶	Rojo Iglesias, Marta	60000.00	2
	Perez Carrillo, Iván	48000.00	2
	Torres Marqués, Fernando	55000.00	2
	Rubio Sánchez, María	36000.00	3
	Roca Benítez, Elena	35000.00	4
	Muñoz Casas, Montse	40000.00	5

Concepto de Vista

*Las vistas son **objetos de las BD** (como las tablas) que almacena la definición de una consulta, por lo que no posee datos propios.*

Concepto de Vista

- Una vista es **el resultado de una consulta SQL** sobre una o varias tablas guardado en forma de tabla, que se puede consultar y actualizar
- Características:
 - Tiene la **misma estructura que una tabla**
 - Es una **tabla virtual** (sólo se guarda la definición, no los datos). No almacena datos físicamente.
 - **Se puede consultar** como cualquier tabla. Consultas a vistas
 - Se le puede insertar, modificar y eliminar datos (hay algunas restricciones)

Aplicaciones de las vistas

- Para la especificación de **tablas con información que se accede con frecuencia**:
 - Información derivada de la relación entre varias tablas
 - Información derivada de consultas complejas
- **Como mecanismo de seguridad**
 - pueden contener únicamente atributos a los cuales se desea permitir acceso a determinados usuarios).
 - Ejemplo: datos públicos y privados de la tabla Empleados
- Para la creación de esquemas externos (diferentes usuarios tienen **diferente visión de la BD**)
- Para mantener la **independencia entre las aplicaciones y las bases de datos**

Creación de vistas

```
CREATE VIEW view_name [ (column[ ,...n ] )  
AS select_statement  
[ WITH CHECK OPTION ] [ ; ]
```

Ejemplo 1: Vista que muestra los trabajadores de Barcelona

```
/*Ejemplo 1  
Creamos una vista con los trabajadores de Barcelona*/  
  
CREATE VIEW trabajadoresbar  
AS SELECT * from trabajadores where ciudad='Barcelona';  
  
Select * from trabajadoresbar;
```

	dni	nombre	ciudad	antiquedad	salario	departamento
►	11112222A	Rojo Iglesias, Marta	Barcelona	12	60000.00	2
	11112266C	Llamas Rocasolano, Isabel	Barcelona	13	28000.00	3
	33112222A	Nualart Vives, Carlos	Barcelona	10	30000.00	4

Creación de vistas

Los nombres de los campos de la vista se heredan de los campos de la/s tabla/s asociadas, pero se pueden cambiar:

```
/*Ejemplo 2
Creamos una vista con los trabajadores de Barcelona
Cambiamos los nombres de los campos*/

CREATE VIEW trabajadoresbar2 (dnit,trabajador,poblacion)
AS SELECT dni,nombre,ciudad from trabajadores where ciudad='Barcelona';

Select * from trabajadoresbar2;
```

	dnit	trabajador	poblacion
▶	11112222A	Rojo Iglesias, Marta	Barcelona
	11112266C	Llamas Rocasolano, Isabel	Barcelona
	33112222A	Nualart Vives, Carlos	Barcelona

Podemos utilizar datos de diversas tablas y realizar todo tipo de consultas.

```
/*Ejemplo 3
Creamos una vista con los datos de varias tablas*/

CREATE VIEW trabajadoresdep
AS SELECT t.*, d.nombredep from trabajadores t, departamento d
where t.departamento=d.numdep;

Select * from trabajadoresdep;

/*Ejemplo 4
Podemos hacer todo tipo de consultas sobre la vista*/

select nombredep, count(*) as total from trabajadoresdep
group by nombredep;
```

	nombredep	total
►	Comercial	2
	Contabilidad	1
	Facturación	1
	Informática	3
	Recursos Humanos	3

	dni	nombre	ciudad	antigüedad	salario	departamento	nombredep
►	22112222A	Roca Benítez, Elena	Sevilla	6	35000.00	4	Comercial
	33112222A	Nualart Vives, Carlos	Barcelona	10	30000.00	4	Comercial
	11112222C	Gomez Corachán, Manuel	Madrid	15	22000.00	1	Contabilidad
	33112222C	Muñoz Casas, Montse	NULL	7	40000.00	5	Facturación
	11112255B	Rubio Sánchez, María	Sevilla	4	36000.00	3	Informática
	11112266C	Llamas Rocasolano, Isabel	Barcelona	13	28000.00	3	Informática
	33224455C	Perez Cano, Isabel	Madrid	2	26000.00	2	Informática

Consulta y eliminación de vistas

Para la obtención de información sobre vistas podemos utilizar la sentencia
SHOW CREATE VIEW

```
SHOW CREATE VIEW trabajadoresdep;
```

	View	Create View	character set client	collation connection
►	trabajadoresdep	CREATE ALGORITHM=UNDEFINED DEFINER=`root` @`localhost` ...	utf8	utf8_general_ci

Para la eliminación de una vista se utiliza la sentencia DROP VIEW

```
DROP VIEW trabajadoresdep
```

- Son actualizables (UPDATE, DELETE). En realidad se actualiza el contenido de la tabla o tablas relacionadas.
- Las vistas actualizables han de cumplir algunas restricciones:
 - No usar cláusulas **GROUP BY** ni **HAVING**
 - Cualquier modificación (UPDATE, INSERT, DELETE) debe hacer referencia a las columnas **de una única tabla**.
 - En el caso de INNER JOIN las vistas se pueden actualizar siempre y cuando los campos afectados sean únicamente los de una de las tablas implicadas en el join
 - Los campos que se van a modificar no se pueden obtener con funciones de agregado: AVG, COUNT, SUM, MIN....

Vistas actualizables

- En el caso de INSERT las columnas de la vista han de cumplir:
 - La vista debe tener todos los campos de la tabla relacionada que no tengan indicado un valor por defecto.
 - Los campos de la vista deben hacer referencia campos simples, y no derivados.
 - No debe haber campos con nombres duplicados.

```
/* Ejemplo 5. Podemos agregar información siempre que llenemos
todos los campos no nulos */
-- Añadimos un registro a la vista trabajadores barcelona.

select * from trabajadoresbar;
INSERT INTO trabajadoresbar VALUES ("33332211C","Oliveras Garcia, Miguel","Barcelona",4, 40000, 2);

-- Comprobamos que ha funcionado.
select * from trabajadoresbar;

-- Comprobamos en la tabla original
select * from trabajadores;
```

	dni	nombre	ciudad	antiquedad	salario	departamento
▶	11112222A	Rojo Iglesias, Marta	Barcelona	12	60000.00	2
	11112266C	Llamas Rocasolano, Isabel	Barcelona	13	28000.00	3
	33112222A	Nualart Vives, Carlos	Barcelona	10	30000.00	4
	33332211C	Oliveras Garcia, Miguel	Barcelona	4	40000.00	2

- WITH CHECK OPTION:

Nos permite insertar sólo aquellos registros que cumplen la condición del WHERE.

```
/*Ejemplo 6
Creamos una vista con los trabajadores de Contabilidad
y hacemos que solo se puedan insertar trabajadores de contabilidad
with check option*/

CREATE VIEW trabajadorescont
AS SELECT * from trabajadores where departamento=1
WITH CHECK OPTION;

select * from trabajadorescont;
-- Añadimos un registro del departamento 2 y nos da error
INSERT INTO trabajadorescont VALUES ("3333222F","Olalla Burgos, Anna","Madrid",6, 45000, 2);

-- Añadimos un registro del departamento 1 y funciona
INSERT INTO trabajadorescont VALUES ("3333222F","Olalla Burgos, Anna","Madrid",6, 45000, 1);

-- Comprobamos.
select * from trabajadorescont;
```

VALUES ("33332211C","Olalla Burgos, Anna","Madrid",6, 45000, 2) Error Code: 1369. CHECK OPTION failed 'empresa.trabajadorescont'

	dni	nombre	ciudad	antiquedad	salario	departamento
▶	11112222C	Gomez Corachán, Manuel	Madrid	15	22000.00	1
	33332222F	Olalla Burgos, Anna	Madrid	6	45000.00	1

Vistas con múltiples tablas

Vista con la BD Escuela accediento a las 4 tablas de golpe.

Simplifica las consultas posteriores (sobre todo desde programación)

```
/* Ejemplo escuela
Podemos juntar los datos de varias tablas, en este caso 4
para que luego las consultas sean más fáciles */

create view alumnosnotas as
select a.numexp, a.nombre, a.ciudad, a.edad, n.semestre, n.nota, s.codiasig,
s.nombre as "nombreasig", s.aula, p.dniprofe, p.nombre as "nombreprofe"
from alumnos a, notas n, asignaturas s, profesores p
where a.numexp=n.numexp AND n.codiasig=s.codiasig AND s.dniprofe=p.dniprofe;

/* Para hacer esta misma consulta sin la vista necesitamos tres tablas*/

select * from alumnosnotas
where nota<5 and nombreasig="inglés";
```

	numexp	nombre	ciudad	edad	semestre	nota	codiasig	nombreasig	aula	dniprofe	nombreprofe
►	0002	Perez Carrillo, Iván	Bilbao	20	1617-1	4.50	ING1E	Inglés	12	11112222A	Valle Roca, Ramon
	0005	Llamas Rocasolano, Isabel	Barcelona	25	1617-1	4.00	ING1E	Inglés	12	11112222A	Valle Roca, Ramon
	0009	Villa Bueno, Sandra	Sevilla	32	1617-1	2.30	ING1E	Inglés	12	11112222A	Valle Roca, Ramon

¡Gracias!

A large, abstract pink graphic on the right side of the slide. It consists of a rounded rectangular shape at the top, a diagonal line extending from the bottom left towards the top right, and a vertical line on the right side, creating a stylized, modern shape.